

Design Specification

Ahmed Fawaizat Oyindamola (fawah595)
Abdul Gaffar Abubakar Passum (abuab954)
Varun Gurupurandar (vargu125)
Mehran Mamivand (mehma172)

February 11, 2026

Version 1.0



Status

Reviewed	Firstname Lastname	date
Approved	Firstname Lastname	date

Project Identity

Group E-mail: tsks23-group3@liu.se

Orderer & Customer: Danyo Danev
Phone: +46 (0)13 28 13 35
E-mail: danyo.danev@liu.se

Supervisor: Oksana Moryakova
Phone: +46 (0)13-28 15 53
E-mail: oksana.moryakova@liu.se

Supervisor: David Nordlund
Phone: +46 (0)13-28 19 53
E-mail: david.nordlund@liu.se

Supervisor: Henrik Åkesson
Phone: +46 (0)13-28 28 95
E-mail: henrik.akesson@liu.se

Course Responsible: Danyo Danev
Phone: +46 (0)13-28 13 35
E-mail: danyo.danev@liu.se

Participants of the group

Name	Role	E-mail
Fawaizat Ahmed	Project Lead	fawah595@student.liu.se
Abdul Gaffar Abubakar Passum	Design Lead	abuab954@student.liu.se
Varun Gurupurandar	Test Lead	vargul25@student.liu.se
Mehran Mamivand	Documentation Lead	mehma172@student.liu.se

CONTENTS

1	Introduction	1
2	System Description	1
3	Input Handling System	3
3.1	File Based Input	3
3.2	Real-Time Audio Capture	3
3.3	Preprocessing	3
3.3.1	Normalization	4
3.3.2	High-Pass Filtering	4
4	Graphical User Interface	4
4.1	Primary Interface Components	4
4.1.1	File Processing Tab	4
4.1.2	Real-Time Processing Tab	4
4.1.3	Results Display Tab	5
4.1.4	History Management Tab	5
4.2	Visualisation Components	5
4.2.1	Waveform Display	5
4.2.2	Spectrogram Visualisation	6
5	Backend (Core Processing Engine)	6
5.1	Processing Pipeline	6
5.2	Onset and Offset Detection	7
5.3	Pitch Detection (YIN)	7
5.4	Frequency-to-Note Mapping	8
5.5	Event Post-Processing	8
6	Database Management System	8
6.1	Schemas	8
7	Output and Export Module	9
7.1	Transcription Output Formats	9
7.1.1	Human-Readable Text Output	9
7.2	Visualisation Outputs	10
7.2.1	Waveform with Note Annotations	10
7.2.2	Spectrogram with Pitch Tracking	10
7.3	Exports and History	10

DOCUMENT HISTORY

Version	Date	Changes made	Sign	Reviewer
0.1	03-11-2025	Initial design draft	AF, AG, VG, MM	DA
0.2	17-11-2025	Updated module breakdown and architecture	AF, AG, VG, MM	DA
1.0	05-12-2025	Final version aligned with implementation	AF, AG, VG, MM	DA

1 INTRODUCTION

This Design Specification describes the architecture and internal structure of the **Automatic Music Transcription System** developed in the TSKS23 CDIO project. The document refines the approved Requirements Specification and explains how those requirements are realised in software and hardware.

The system converts acoustic piano audio (and MIDI keyboard input) into structured musical information. It detects onsets, offsets, pitches and durations and presents the result as human-readable note lists and stored database records. The design targets:

- **Input sources:** audio files (e.g. WAV, MP3) and real-time microphone or USB-MIDI keyboard recordings.
- **Instruments:** 61-key piano keyboard in the range C2–C7.
- **Signal quality:** fixed sampling rate of 40 960 Hz for all audio processing.

The system is implemented in Python, using Librosa [1] for signal processing, PyAudio for streaming audio capture, PyQt5 for the graphical user interface, and SQLite for persistent storage. The design is modular and is divided into five main subsystems:

1. Input Handling System
2. Graphical User Interface (GUI)
3. Backend Core Processing Engine
4. Database Management System
5. Output and Export Module

Each of these modules is described in detail in the following sections, including their responsibilities, internal structure, key algorithms and interfaces to other modules.

2 SYSTEM DESCRIPTION

The Music Transcription System automates the conversion of acoustic audio into structured musical information, capturing pitch, onset time, duration and note identity for each detected event. It supports both file-based and real-time inputs, applies preprocessing, onset/offset detection and pitch estimation, and stores the resulting transcription together with metadata.

Key features are:

- **Input sources:** file upload (WAV, MP3, FLAC, OGG) and real-time recording via microphone or USB-MIDI keyboard.
- **Preprocessing:** peak normalization and high-pass filtering to remove low-frequency rumble.
- **Core engine:** onset detection based on Librosa's onset strength envelope and tuned peak picking, and pitch detection based on the YIN algorithm [2] applied to short, high-energy segments.
- **Output:** waveform and spectrogram visualisation, note listings, and TXT exports; all transcriptions are stored in an SQLite database for later retrieval.
- **GUI:** a tabbed PyQt5 interface providing modes for file processing, real-time recording, results inspection and history management.

The system targets 61-key keyboard input (C2–C7) and uses a fixed sampling rate of 40 960 Hz to provide sufficient frequency resolution for piano fundamentals and overtones.

The overall modular structure of the system is illustrated in Figure 1. The diagram shows how user input is acquired and processed by the backend transcription engine, which acts as the core logic layer of the system. The backend performs signal analysis and transcription, and then forwards structured musical data for presentation and storage.

The arrow from the backend to the GUI represents the *data output path*, through which transcription results, waveform data, and spectrograms are sent for visualisation and user inspection. In parallel, the backend stores and retrieves information from the database to ensure persistent storage of audio metadata, transcription results, and processing logs.

In addition, the direct line between the GUI and the backend represents a *bidirectional control and status communication channel*. Through this connection, the GUI sends user commands and runtime parameters to the backend (e.g. start/stop recording, file selection, transcription settings), while the backend reports processing progress, warnings, errors, and execution status back to the GUI. This explicit separation highlights that the system is interactive rather than a one-way batch processing pipeline.

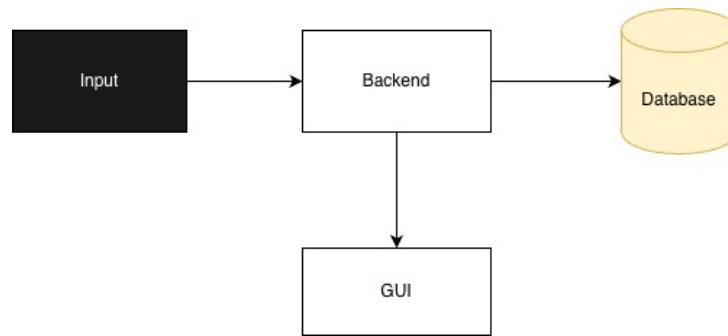


Figure 1: Overall Music Transcription System flow.

The internal interfaces between the GUI, backend, database, and external input and output components are further detailed in Figure 2. The backend functions as an intermediary between the user interface and the database, ensuring a clean separation between presentation, application logic, and data persistence. This modular organisation supports independent development, testing, and future extension of each subsystem.

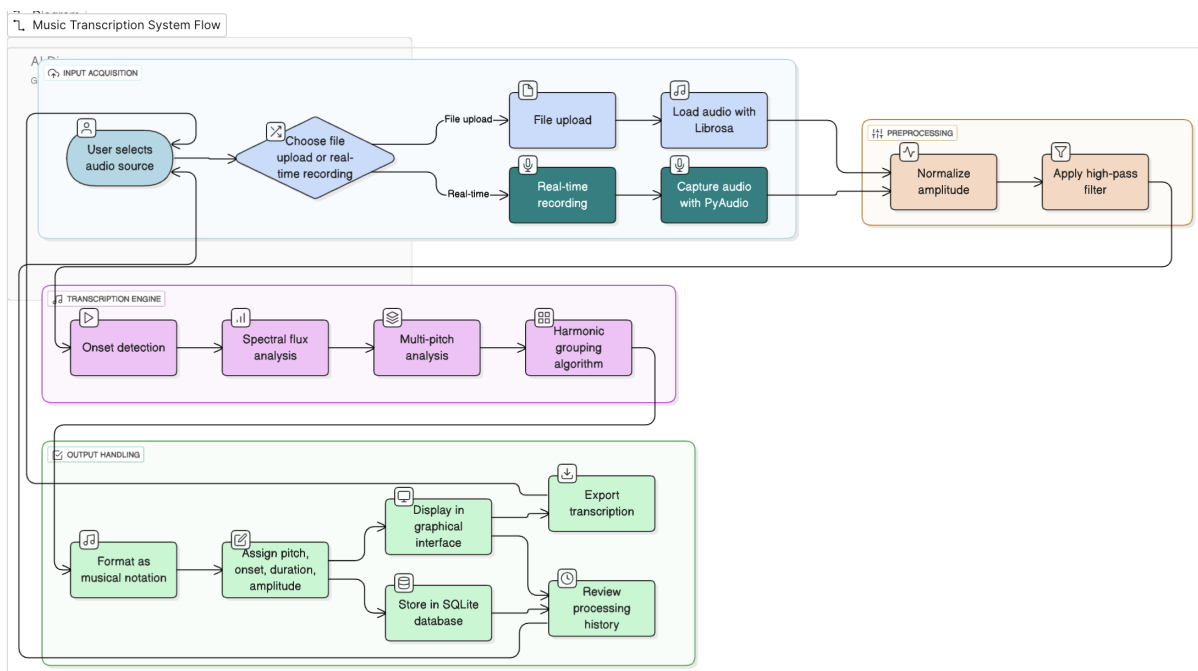


Figure 2: High-level system architecture showing GUI, backend, database, input and output interactions.

3 INPUT HANDLING SYSTEM

The input handling module is the system's sensory gateway. It collects and prepares audio data for downstream processing and supports two primary modes: *batch processing* of prerecorded files and *streaming capture* from live sources. Both modes are designed for fidelity and predictable timing.

3.1 File Based Input

For file based inputs, the module employs Librosa's loader, which decodes common audio formats such as WAV and MP3 while resampling to a standard sampling rate of 44 100 Hz. This uniform rate aligns the temporal resolution with the target piano range (approximately 65 Hz for C2 up to 2–4 kHz for higher harmonics). After loading, the audio array y and sample rate sr are returned together with the duration

$$T = \frac{\text{len}(y)}{sr},$$

which is used throughout the system for time axis generation and segment slicing. Errors (e.g. corrupted files) are trapped and returned as user-friendly messages instead of causing a crash.

```
1 def load_audio(self, file_path):
2     """Load audio file using librosa with error handling"""
3     try:
4         y, sr = librosa.load(file_path, sr=self.sample_rate)
5         duration = len(y) / sr
6         return y, sr, duration
7     except Exception as e:
8         raise Exception(f"Error loading audio file: {str(e)}")
```

Listing 1: Audio loading implementation

3.2 Real-Time Audio Capture

Real-time input is managed by the `RecordingManager` using `PyAudio`. The stream configuration matches the offline pipeline:

- **Sample rate:** 44 100 Hz
- **Channels:** mono
- **Format:** 16-bit PCM (`paInt16`)
- **Chunk size:** 2 048 samples per buffer

`PyAudio` invokes a callback that appends each buffer to an internal list while recording is active. When the user stops recording, the accumulated frames are converted to a NumPy array of `float32` values in the range $[-1, 1]$.

```
1 self.audio_stream = self.pyaudio_instance.open(
2     format=pyaudio.paInt16,
3     channels=1,
4     rate=sample_rate,
5     input=True,
6     frames_per_buffer=chunk_size,
7     stream_callback=self.audio_callback
8 )
```

Listing 2: Real-time audio configuration

3.3 Preprocessing

All input signals pass through a lightweight preprocessing chain to make the subsequent onset and pitch detectors more robust across different recording conditions.

3.3.1 Normalization

The audio signal is first normalised to unit peak amplitude:

$$y_{\text{norm}}[n] = \frac{y[n] - \mu_y}{\max(|y[n] - \mu_y|)}, \quad (1)$$

where μ_y is the mean of $y[n]$. This removes overall gain variations and reduces the risk of clipping.

3.3.2 High-Pass Filtering

A simple first-order high-pass filter suppresses low-frequency rumble from room noise and handling:

$$y_{\text{filtered}}[n] = y_{\text{norm}}[n] - \alpha y_{\text{norm}}[n-1], \quad (2)$$

with $\alpha = 0.97$, corresponding to a cutoff around 80 Hz. This keeps the piano fundamentals and harmonics largely unaffected while stabilising the energy envelope used in onset detection.

The resulting, cleaned waveform is forwarded to the backend processing engine and to the GUI for waveform and spectrogram visualisation.

4 GRAPHICAL USER INTERFACE

The graphical user interface (GUI) module provides the user-facing interface implemented in PyQt5. It uses a dark-themed tabbed layout to organise the main workflows into four modes:

- **Transcribe (Real-Time)** for live microphone/keyboard capture.
- **Transcribe (File)** for processing uploaded audio files.
- **Results** for inspecting note lists and visualisations.
- **History** for browsing previously stored sessions.

Core widgets include `QPushButton` controls for actions (record, stop, process, export), Matplotlib-embedded canvases for waveform and spectrogram plots, and `QTextEdit` fields for textual output and logs. The GUI communicates bidirectionally with the backend using signal/slot connections so that long operations can report progress and errors via `QMessageBox`.

4.1 Primary Interface Components

4.1.1 File Processing Tab

- Audio file browser with basic format filtering.
- Real-time waveform and spectrogram visualisation after loading.
- Configuration of maximum simultaneous notes for transcription.
- Processing controls with progress feedback and status messages.

4.1.2 Real-Time Processing Tab

- Live audio level indication during recording.
- Start/stop recording buttons bound to the `RecordingManager`.
- Live waveform display during capture.
- Automatic transcription triggered when recording stops.

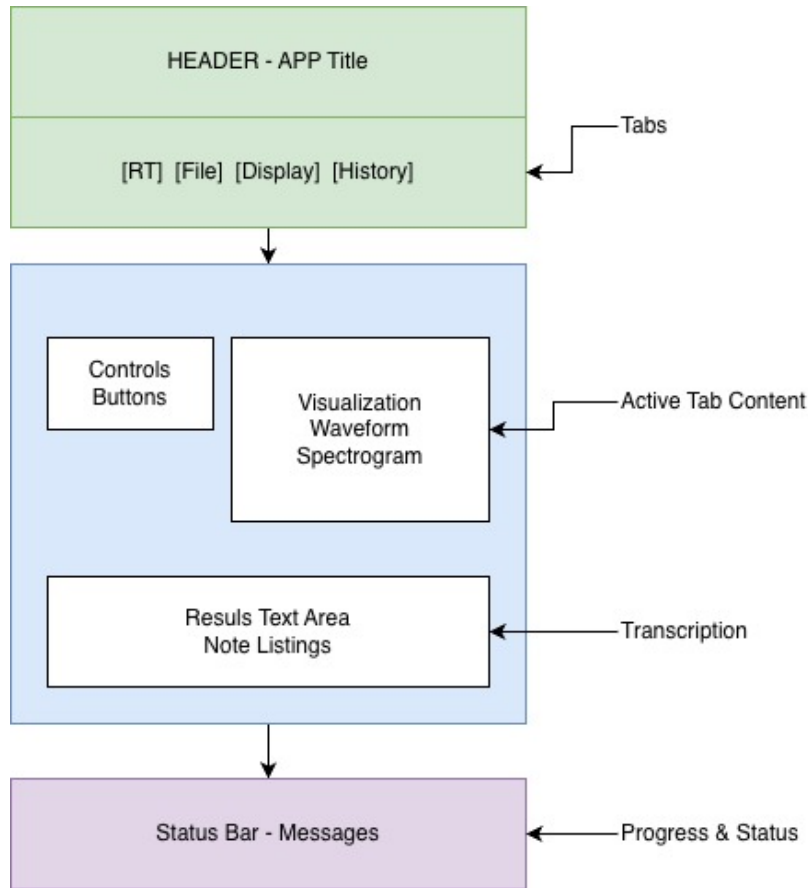


Figure 3: Block diagram of the user interface.

4.1.3 Results Display Tab

- Formatted note transcription display (onset, offset, duration, note).
- Timeline visualisation with note annotations on the waveform.
- Export functionality for human-readable TXT files.
- Processing logs with timestamps for debugging.

4.1.4 History Management Tab

- Database-driven session list (date, filename, duration).
- Search and filter capabilities.
- Reloading of stored transcriptions into the Results tab.

4.2 Visualisation Components

4.2.1 Waveform Display

The waveform view shows the amplitude A_i versus time t_i :

$$\text{Waveform} = \{(t_i, A_i) \mid t_i = \frac{i}{f_s}, A_i = y[i]\}, \quad (3)$$

where f_s is the sampling rate and $y[i]$ is the audio amplitude.

4.2.2 Spectrogram Visualisation

The spectrogram is computed as

$$\text{Spectrogram}(t, f) = 20 \log_{10} |\text{STFT}(t, f)|, \quad (4)$$

using the Short-Time Fourier Transform for time-frequency representation.

```
1 def compute_spectrogram(self, audio_data, sr):
2     """Compute spectrogram using STFT"""
3     stft = librosa.stft(audio_data)
4     spectrogram = librosa.amplitude_to_db(np.abs(stft), ref=np.max)
5     return stft, spectrogram
```

Listing 3: Spectrogram computation

5 BACKEND (CORE PROCESSING ENGINE)

The core processing engine is responsible for converting cleaned audio into musical events. It implements the main analysis pipeline:

1. Onset and offset detection.
2. Pitch estimation for each note segment.
3. Mapping of frequencies to musical notes.
4. Post-processing and packaging of events for output and storage.

The implementation closely follows the requirements on timing accuracy and pitch range while remaining computationally lightweight.

5.1 Processing Pipeline

The pipeline operates on a one-dimensional mono waveform $y[n]$ sampled at $f_s = 40\,960$ Hz:

1. **Onset strength computation** using Librosa's onset envelope.
2. **Peak picking** with tuned parameters to determine onset times.
3. **Offset estimation** from the next onset or end of file, with a minimum note duration.
4. **Pitch estimation** on short, high-energy segments using YIN.
5. **Note mapping** from frequency (Hz) to MIDI and note name.
6. **Event post-processing** (removal of very short notes and merging of nearly contiguous identical notes).

The sequence of processing stages implemented in the backend is summarised in Figure 4.



Figure 4: Audio processing pipeline in the core engine.

5.2 Onset and Offset Detection

Onset detection uses Librosa's onset strength envelope computed from the spectrogram. The envelope is lightly smoothed with a moving average, and peaks are detected with custom peak-picking parameters that were tuned on the project's piano datasets.

The detector operates at hop length $H = 1024$ samples and uses backtracking to align onsets with preceding spectral changes:

```

1 onset_env = librosa.onset.onset_strength(
2     y=audio_data,
3     sr=sr,
4     hop_length=self.hop_length
5 )
6
7 # simple smoothing
8 if len(onset_env) >= 3:
9     kernel = np.ones(3) / 3.0
10    onset_env = np.convolve(onset_env, kernel, mode="same")
11
12 onset_times = librosa.onset.onset_detect(
13     onset_envelope=onset_env,
14     sr=sr,
15     hop_length=self.hop_length,
16     backtrack=self.backtrack,
17     units="time",
18     pre_max=3,
19     post_max=3,
20     pre_avg=3,
21     post_avg=3,
22     delta=0.06,
23     wait=1
24 )

```

Listing 4: Onset detection configuration

Detected onsets that are closer than a configurable minimum gap ($\text{min_gap} = 0.05$ s) are merged so that repeated triggers on the same key press are treated as a single note. For each onset t_i the corresponding offset is defined as the next onset t_{i+1} or, for the last note, the audio end. A minimum note duration of $\text{min_note_duration} = 0.08$ s is enforced:

$$t_{\text{offset},i} = \min(\max(t_{i+1}, t_i + T_{\min}), T_{\text{audio}}), \quad (5)$$

where $T_{\min}$ is the minimum duration and T_{audio} is the total audio length.

5.3 Pitch Detection (YIN)

Pitch detection is performed by the `EnhancedPitchDetector`. For each segment between onset and offset, only the first 0.5 s (or less, if the segment is shorter) is analysed to focus on the stable part of the note and avoid the noisy tail. Very low-energy segments ($\text{RMS} < 0.005$) are discarded as likely silence:

```

1 max_duration = 0.5
2 max_samples = int(max_duration * self.sample_rate)
3 analysis_segment = audio_segment[:max_samples]
4
5 rms = np.sqrt(np.mean(analysis_segment**2))
6 if rms < 0.005:
7     return []
8
9 f0 = librosa.yin(
10     analysis_segment,
11     fmin=self.fmin,
12     fmax=self.fmax,
13     sr=self.sample_rate
14 )
15 f0 = f0[~np.isnan(f0)]
16 if len(f0) == 0:
17     return []

```

```

18
19 main_f0 = float(np.median(f0))
20 return [main_f0]

```

Listing 5: YIN-based pitch estimation

The detector is configured for the piano range C2–C7 and returns a list of dominant pitches in Hz. In the current prototype, at most two simultaneous pitches per note segment are supported, which already covers simple chords and octave doublings.

5.4 Frequency-to-Note Mapping

The `NoteMapping` class converts frequencies to MIDI note numbers and human-readable note names assuming equal temperament and $A4 = 440$ Hz. A frequency f_0 is mapped to MIDI as

$$m = 69 + 12 \log_2 \left(\frac{f_0}{440} \right), \quad (6)$$

rounded to the nearest integer. Only notes in the configured range ($m \in [36, 96]$ corresponding to C2–C7) are accepted. The MIDI number is then decomposed into a pitch class and octave.

For each segment, the median of all valid f_0 values is used as the final pitch estimate, which is robust against occasional errors in the YIN track.

5.5 Event Post-Processing

The `TranscriptionEngine` gathers onset times, offsets and pitch estimates into a list of note events:

- onset time (s)
- offset time (s) and duration
- list of detected pitches (Hz) and corresponding note names

A final clean-up step removes extremely short notes and merges nearly contiguous events with identical note content and a small time gap (< 0.05 s). This prevents artificial note fragmentation due to small fluctuations in the onset detector and yields a compact, musically meaningful transcription ready for output and storage.

6 DATABASE MANAGEMENT SYSTEM

The database module employs SQLite for persistent storage, with tables for `audio_files` (metadata: ID, filename, path, duration, rate, timestamp), `transcriptions` (per-event: onset, offset, duration, note count, linked by foreign key), and `processing_logs` (events: level, message, timestamp). An additional table `transcription_notes` stores per-note information for polyphonic events. This one-to-many relational schema (Figure 5) supports querying history, exporting results and logging for auditing.

6.1 Schemas

The schema revolves around four interconnected tables. The `audio_files` table stores metadata—filename, path, duration, sample rate and upload timestamp—serving as the root for foreign key cascades. Linked via `audio_file_id`, the `transcriptions` table logs event-level data: onset/offset times, durations and note counts. Granular details reside in `transcription_notes`, which enumerates per-event pitches, note names (e.g. “C4”) and ordinal positions within chords. Finally, `processing_logs` chronicles diagnostics: INFO for milestones and ERROR for anomalies, each with timestamps.

Insert operations are wrapped in transactions to minimise I/O overhead, while queries leverage indexes on timestamps and IDs for fast retrieval. For example, reconstructing a session involves a join:

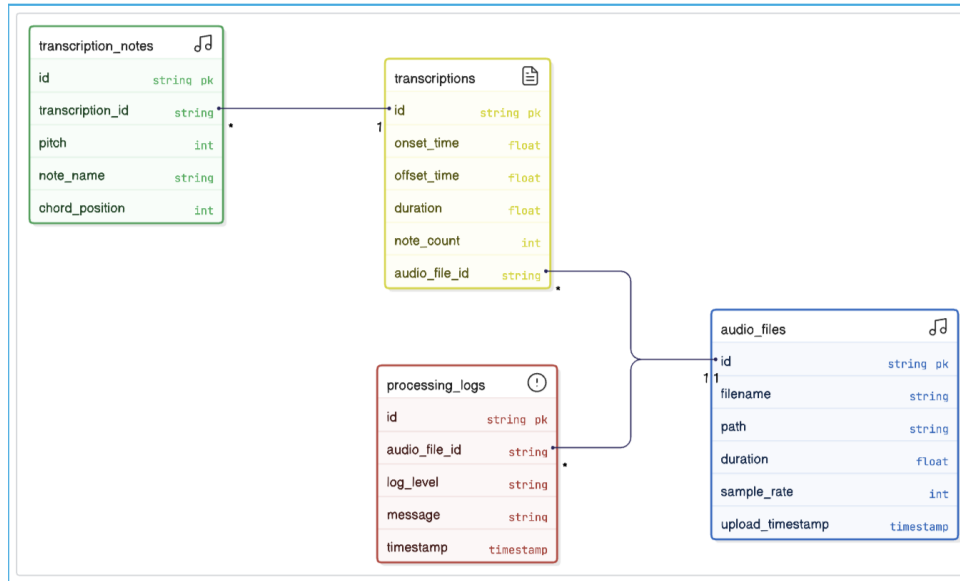


Figure 5: Database schema for audio files, transcriptions, notes and processing logs.

```

SELECT *
FROM transcriptions t
JOIN transcription_notes n
  ON t.id = n.transcription_id
WHERE t.audio_file_id = ?
ORDER BY t.onset_time;
    
```

This design ensures data integrity (e.g. non-negative durations) and enables later analytics such as accuracy statistics or user-specific performance summaries.

7 OUTPUT AND EXPORT MODULE

The output and export module generates user-consumable results from the internal transcription representation. It is responsible for:

- Formatted textual note listings.
- Graphical displays (waveform and spectrogram with annotations).
- Export of results to external files.
- Forwarding events to the database module for persistent storage.

7.1 Transcription Output Formats

7.1.1 Human-Readable Text Output

The primary human-readable format is a tabular plain-text summary that can be displayed in the GUI or saved to file:

Multi-Pitch Transcription Results:

```

=====
Total note events: 15
Chord events (multiple notes): 3
    
```

```
Note 1: C4          | Onset: 0.123s | Offset: 0.579s | Duration: 0.456s | Pitch: 261.6 Hz
Chord 2: C4 + E4    | Onset: 1.234s | Offset: 1.789s | Duration: 0.555s | Notes: 2
Note 3: G4          | Onset: 2.345s | Offset: 2.891s | Duration: 0.546s | Pitch: 392.0 Hz
```

This format mirrors the internal event structure (onset, offset, duration and per-note pitch list) and can be parsed again if needed.

7.2 Visualisation Outputs

7.2.1 Waveform with Note Annotations

The system generates annotated waveforms showing detected note regions:

$$\text{Annotation}(t) = \begin{cases} \text{Note colour,} & t_{\text{onset}} \leq t \leq t_{\text{offset}}, \\ \text{Transparent,} & \text{otherwise,} \end{cases} \quad (7)$$

where each colour corresponds to a specific event. This allows users to check if onsets and offsets align with the audible notes.

7.2.2 Spectrogram with Pitch Tracking

Pitch tracks can be overlaid on the spectrogram for visual verification:

$$\text{PitchTrack} = \{(t_i, f_i) \mid f_i \text{ is the detected pitch at time } t_i\}. \quad (8)$$

7.3 Exports and History

For persistence and later analysis, all transcription events and associated metadata are written to the SQLite database through the database module. Additionally, the GUI provides an export dialog (based on `QFileDialog`) for saving the current transcription as a TXT file. This combination of structured storage and lightweight export supports both iterative development and final demonstration use cases.

REFERENCES

- [1] B. McFee, C. Raffel, D. Liang, *et al.*, “librosa: Audio and music signal analysis in python,” in *Proceedings of the 14th Python in Science Conference*, 2015.
- [2] A. De Cheveigné and H. Kawahara, “Yin, a fundamental frequency estimator for speech and music,” *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.